개발환경 구성

Git · venv · pip · VSCode

cchyun@gmail.com

Part 1. Git 필요성과 핵심 개념

Git이란 무엇인가? — "코드 고고학"

- **Git은 코드에 가해진 모든 수정 사항을 추적하여 프로젝트의 역사를 기록하는 버전 관리 시스템**
  - **"코드 고고학(Archaeology of code)"**
    - 역사 유적을 발굴하듯, 코드의 변경 이력을 발굴·탐색 가능
  - **"정교한 게임 세이브 시스템"**
    - 단순 파일 저장이 아닌, 특정 시점의 모든 환경과 로직을 **스냅샷**으로 보존
    - 어려운 구간(복잡한 알고리즘) 직전, 안정 상태에서 저장 가능
  - **브랜치(Branching) = 멀티 엔딩 테스트**
    - A선택 / B선택을 **별도 세이브 파일**처럼 독립 실험
    - 기존 안정 코드를 건드리지 않고 새 기능을 실험

Git 없이 개발할 때 생기는 문제

- **버전 관리 시스템이 없다면 개발자는 다음과 같은 상황에 직면**
  - **"최종_진짜최종_내일제출_v12.py"의 늪**
    - 어떤 파일이 최신인지, 어떤 기능이 어떤 버전에 있는지 파악 불가
  - **협업 중 "코드 덮어쓰기" 대참사**
    - 여러 사람이 동시에 같은 파일 수정 → 나중 저장자의 코드가 이전 작업을 완전히 삭제
    - Git은 이런 충돌 방지 + 안전한 통합(Merge) 지원
  - **"어제는 됐는데 왜 오늘은 안 되지?"의 공포**
    - 어떤 수정이 문제인지 일일이 대조해야 함
    - Git이 있다면 **한 줄 명령어로 과거 안정 상태로 즉시 복구(Revert)**

로봇 개발 현장: Git 없을 때 사고 시나리오

- **로봇 SW는 물리 하드웨어와 직접 상호작용 → Git 부재가 실제 하드웨어 파손으로 이어질 수 있음**
  - **시나리오 A: PID 제어 게인값 유실 사고**
    - 며칠간 최적의 PID 게인값을 튜닝
    - 다음 날 다른 버그 수정 중 실수로 제어 로직 파일을 덮어씌움
    - 대략적인 값으로 재실행 → 로봇이 진동(Oscillation)하며 충돌 → **모터 소손**
  - **시나리오 B: 검증되지 않은 알고리즘의 메인 시스템 침범**
    - 새 Lidar 필터 알고리즘을 브랜치 없이 기존 주행 코드에 직접 수정
    - 예기치 못한 에러 발생 → **비상 정지(Emergency stop) 로직까지 마비**
    - 로봇이 장애물을 인식하지 못하고 전속력으로 벽에 충돌

Git의 3단계 핵심 영역

- **Git의 작동 원리: 세 가지 영역의 흐름을 이해해야 함**
  - **스테이징 영역 (Staging Area) — "준비물 바구니"**
    - `git add` 로 커밋에 포함할 변경 사항을 선택적으로 담는 공간
    - PID 튜닝 기록만, 또는 버그 수정만 선별하여 기록 가능
  - **로컬 저장소 (Local Repository) — "개인 코드 저장고"**
    - `git commit` 으로 확정하면 해당 시점의 스냅샷이 영구 기록
    - 인터넷 연결 없이도 전체 히스토리 탐색·복구 가능
  - **원격 저장소 (Remote Repository, GitHub 등) — "공유 저장고"**
    - `git push` 로 업로드 → 팀원 공유 가능
    - **Pull Request** 를 통한 동료 검토(Peer-review) 후 최종 병합

입문자가 가장 혼란스러워하는 포인트

- **"파일 저장(Save)과 커밋(Commit)은 뭐가 다른가요?"**
  - 파일 저장: 하드디스크에 현재 내용을 기록하는 단순한 동작
  - **커밋: 의미 있는 작업 단위의 "역사적 기록"을 만드는 것**
  - 가이드: "로봇이 한 단계를 성공했을 때만 커밋" → '센서 읽기 성공', '모터 구동 완성'
- **"add는 왜 하는 건가요? 바로 commit 하면 안 되나요?"**
  - **장보기 비유:**
    - 물건을 집을 때마다 결제(Commit)하지 않고, 카트(Staging Area)에 담았다가 한꺼번에 결제
    - 잘못 담은 물건은 카트에서 꺼내면 그만
    - 이미 결제한 물건의 환불(Revert)은 절차가 복잡 → **신중한 기록의 중요성**

Part 2. Git 핵심 명령어

git init — 저장소의 시작

- **새로운 프로젝트를 시작할 때 로컬 프로젝트 저장소를 초기화하는 명령어**

  - **내부 동작**
    - 현재 디렉토리에 `.git`**이라는 숨겨진 폴더** 생성
    - 이 폴더가 프로젝트의 모든 변경 이력과 메타데이터를 담는 저장소 역할

  - **터미널 출력 예시:**

```
$ git init
Initialized empty Git repository in /home/user/Workspace/robot_project/.git/
```

  - **자주 막히는 부분**
    - 폴더를 잘못 지정하여 엉뚱한 곳에 `.git`이 생성되는 경우
    - 이미 저장소인 폴더 안에서 또 `git init`을 실행하지 않도록 주의

git add — 스테이징 영역(Staging Area)

- **작업 디렉토리의 변경 내용을 스테이징 영역으로 이동시키는 단계**

  - **실제 의미**
    - 다음 커밋을 수행하기 전의 **'준비 단계'**
    - 모든 변경 사항을 한꺼번에 저장하는 것이 아닌, **의미 있는 단위로 선택적 스테이징**

```
$ git add controller.py
$ git status
On branch main
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
        modified:   controller.py
```

  - **자주 막히는 부분**
    - `git add` 후 파일을 수정하고 바로 커밋하면 → **add 시점의 상태만 저장됨**
    - 수정 후에는 반드시 다시 `add` 를 해주어야 함

git commit — 스냅샷(Snapshot) 기록

- **프로젝트의 현재 상태를 메시지와 함께 스냅샷으로 저장하는 과정**

  - **스냅샷의 의미**
    - 특정 시점의 전체 프로젝트 상태를 하나의 사진처럼 찍어 기록
    - 커밋 메시지에는 **변경의 근거(rationale)** 를 담아야 함
    - 예: "PID 게인 값 튜닝", "센서 타임아웃 버그 수정"

```
$ git commit -m "Fix sensor timeout bug in lidar module"
[main 4f2a3b1] Fix sensor timeout bug in lidar module
1 file changed, 5 insertions(+), 2 deletions(-)
```

  - **자주 막히는 부분**
    - 메시지 없이 커밋 시도 → 익숙하지 않은 텍스트 에디터(Vim)가 열려 당황
    - `-m` **옵션으로 메시지를 직접 입력하는 습관 권장**

git push — 로컬과 원격의 동기화

- **로컬 저장소의 커밋들을 GitHub 같은 원격 저장소로 업로드하는 과정**

  - **동기화 방식**
    - 분산 버전 관리 특성상, 모든 팀원은 전체 히스토리의 복사본 보유
    - `push` 로 서버에 반영 → 다른 팀원이 내 작업을 내려받아 협업 가능

```
$ git push origin main
Enumerating objects: 5, done.
Writing objects: 100% (3/3), 320 bytes | 320.00 KiB/s, done.
To github.com:user/robot_project.git
4f2a3b1..7e8d9c2  main -> main
```

  - **자주 막히는 부분**
    - **Non-fast-forward 에러:** 원격에 내가 없는 새로운 커밋이 이미 있을 때 발생
      → `git pull` 로 먼저 변경사항을 가져온 후 다시 `push`
    - **Merge Conflict:** 여러 사람이 같은 파일의 같은 부분 수정 시 발생
      → 소스 코드 내 충돌 지점을 직접 확인하고 수정

GitHub 원격 저장소 초기화 후 첫 Push

- **로컬 저장소를 GitHub에 처음 연결하는 절차** (1/2)

  - **① GitHub에서 새 저장소 생성**
    - github.com → **New repository** → 이름 입력 → **Create repository**
    - `README`, `.gitignore` 추가 없이 **빈 저장소**로 생성해야 충돌 방지

  - **② SSH 키 생성 및 GitHub 등록** *(최초 1회)*

```
ssh-keygen -t ed25519 -C "your_email@gmail.com"   # 키 생성 (Enter 3번)
cat ~/.ssh/id_ed25519.pub                          # 공개키 출력 후 복사
```

    - GitHub → **Settings** → **SSH and GPG keys** → **New SSH key** → 붙여넣기 후 저장

GitHub 원격 저장소 초기화 후 첫 Push

- **로컬 저장소를 GitHub에 처음 연결하는 절차** (2/2)

  - **③ 로컬 저장소에 원격 주소 등록 (SSH 방식)**
    - GitHub 저장소 페이지 → **Code** → **SSH** 탭에서 주소 복사

```
# git@github.com:{GitHub유저명}/{저장소명}.git
git remote add origin git@github.com:paul-hyun/robot_project.git
```

  - **④ 기본 브랜치 이름 설정 후 첫 Push**

```
git branch -M main
git push -u origin main
```

    - `-u` 옵션: 이후부터는 `git push` 만 입력해도 `origin main` 으로 자동 연결
    - SSH 키 등록 후 **비밀번호/토큰 없이 자동 인증**

Part 3. Git 자주 하는 실수 TOP 10

실수 1~3: 환경 파일 관련

- **실수 1: 가상환경(`.venv`) 폴더를 저장소에 포함**
  - `git add .` 실행 시 수천 개의 라이브러리 파일이 함께 스테이징 → 저장소 수백 MB로 팽창
  - **해결:** `git rm -r --cached .venv` → `.gitignore`에 `.venv/` 추가
  - **예방:** 프로젝트 시작 직후 반드시 `.gitignore` 먼저 생성

- **실수 2: 대용량 로봇 모델 및 데이터셋 커밋**
  - Gazebo용 STL/DAE 파일, LiDAR 포인트 클라우드 로그 파일을 그대로 커밋
  - `git push` 시 파일 크기 제한 초과 또는 네트워크 저하
  - **해결:** 대용량 파일은 외부 저장소에 두고, Git에는 스크립트/링크만 관리

- **실수 3: `requirements.txt` 업데이트 누락**
  - NumPy, OpenCV 설치 후 정보 기록 없이 코드만 커밋
  - 다른 팀원이 클론 시 `ModuleNotFoundError` 발생
  - **해결:** 패키지 설치 후 `pip freeze > requirements.txt` 실행 후 함께 커밋

실수 4~6: 커밋 관리 문제

- **실수 4: 하드코딩된 로봇 IP 주소 및 설정값 커밋**
  - 로봇 IP, 포트 번호를 코드에 직접 작성 → 다른 환경에서 작동 불가 + 보안 취약
  - **해결:** 설정값은 `.json` / `.env` 파일로 분리 후 `.gitignore`에 추가
  - **예방:** `Config` 클래스로 외부 설정 파일을 동적 로드하는 구조 채택

- **실수 5: `main` 브랜치에서 직접 실험적인 알고리즘 수정**
  - 안정적인 로봇 제어 코드가 있는 `main` 브랜치에서 새 내비게이션 알고리즘 직접 테스트
  - 테스트 오류 발생 시 다른 팀원의 작업까지 중단
  - **해결:** `git branch <new_feature>` 로 새 브랜치 생성 후 검증이 끝나면 머지

- **실수 6: `git status` 확인 없이 모든 변경사항 커밋**
  - `git add .` + `git commit` 을 습관적으로 실행
  - 시뮬레이션 중 생성된 로그, 임시 설정 파일이 의도치 않게 저장소에 포함
  - **해결:** 커밋 전 반드시 `git status` 로 스테이징 파일 목록 검토

실수 7~9: 협업 및 로그 관련

- **실수 7: Pull 하지 않고 Push 시도**
  - 다른 사람이 수정한 코드를 내려받지 않은 상태에서 업로드 시도
  - `non-fast-forward error` 발생으로 푸시 거부
  - **해결:** 작업 시작 전과 커밋 전에 항상 `git pull` 먼저 수행

- **실수 8: 의미 없는 커밋 메시지 작성**
  - "update", "fix", "..." 등 내용 없는 메시지 → 히스토리 파악 불가
  - **해결:** "PID 제어기 게인 값 조정" / "시리얼 통신 타임아웃 오류 수정"처럼 구체적으로 작성
  - '무엇을' 했는지뿐만 아니라 '왜' 했는지 간략히 포함

- **실수 9: 가상환경 활성화 없이 패키지 설치**
  - 시스템 파이썬에 패키지가 설치되어 환경이 꼬임 / 설치했는데 `ModuleNotFoundError`
  - **해결:** 프롬프트에 `(.venv)` 표시 확인 후 없으면 `source .venv/bin/activate` 실행

실수 10: 협업 및 로그 관련

- **실수 10: 시뮬레이션 로그 파일 포함**
  - `.log`, `.bag` 파일을 `.gitignore` 없이 커밋 → 저장소 용량 낭비
  - **해결:** `*.log` 패턴을 `.gitignore`에 등록 / ROS2 권장 `.gitignore` 템플릿 적용

Part 4. Python 가상환경 (venv)

가상환경이 없으면 — 의존성 지옥(Dependency Hell)

- **가상환경 없이 시스템 전역(Global)에만 패키지를 설치할 경우 발생하는 버전 충돌 문제**

  - **사고 시나리오:**
    - **프로젝트 A (로봇 팔 제어):** `NumPy 1.18` 기반 정밀 기하학 계산
    - **프로젝트 B (자율주행 드론):** 최신 기능 포함 `NumPy 1.25` 필요

  - **결과:**
    - 시스템 전역에 NumPy를 최신 버전으로 업데이트하는 순간
    - 프로젝트 A의 코드는 더 이상 작동하지 않음
    - 패키지의 **API가 변경**되거나 기존 함수가 제거됨

  - **결론:**
    - 가상환경 없이는 하나의 프로젝트를 위해 **다른 프로젝트의 안정성을 포기**해야 함

venv 내부 동작 — 경로 조작을 통한 격리

- **venv는 물리적 하드웨어 격리가 아닌, 논리적인 경로 조작으로 환경을 격리**
  - **폴더 구조:**
    - 가상환경 생성 시 프로젝트 폴더 내에 별도 디렉토리(`.venv`) 생성
    - **전용 Python 인터프리터 복사본** + 패키지가 저장되는 `site-packages` 폴더 + `pyvenv.cfg` 포함
  - **PATH 변수 변경:**
    - 활성화 시 쉘의 `PATH` 환경 변수 맨 앞에 가상환경의 실행 폴더(`bin` 또는 `Scripts`) 경로 추가
  - **격리 원리:**
    - `python` / `pip` 입력 시, 시스템 경로보다 가상환경 폴더 내 실행 파일을 먼저 탐색
    - 시스템 표준 설치 경로와 완전히 분리된 독립 공간 형성

venv 생명 주기 — 4단계

- **가상환경의 생성부터 비활성화까지 네 단계로 나뉨**
  - **① 생성** `python -m venv .venv`
    - 지정된 이름의 폴더 생성 + Python 실행 파일 복사/심볼릭 링크 연결
    - 기본 정보를 담은 `pyvenv.cfg` 파일 생성
  - **② 활성화 (Activate)**
    - 활성화 스크립트 실행 → 현재 터미널 세션의 `PATH` 수정
    - 터미널 프롬프트 앞에 `(.venv)` 표시
  - **③ 패키지 설치** `pip install`
    - 활성화된 환경의 `site-packages` 폴더에만 라이브러리 설치
    - 시스템 전역 폴더에는 아무런 영향 없음
  - **④ 비활성화** `deactivate`
    - 수정되었던 `PATH` 원래대로 복구 → 다시 시스템 Python 환경으로 복귀

운영체제별 활성화 명령어 차이

- 운영체제와 쉘 종류에 따라 활성화 명령어 형식이 다름

운영체제	명령어
Windows (CMD)	`.venv\Scripts\activate`
Windows (PowerShell)	`.\.venv\Scripts\Activate.ps1`
macOS / Linux	`source .venv/bin/activate`

입문자가 가장 많이 헷갈리는 포인트 5가지

- **venv 사용 시 자주 발생하는 오해와 실수**

1. **venv 폴더 안에 소스 코드를 넣어야 하는가?**
   - **NO** — 가상환경 폴더는 라이브러리 저장용, 코드는 **폴더 바깥**에 위치

2. **가상환경을 복사해서 옮겨도 되는가?**
   - **절대 안 됨** — 내부에 **절대 경로**가 포함된 스크립트가 있어 폴더 이동 시 경로 파손

3. **가상환경을 Git에 올려야 하는가?**
   - **NO** — 용량이 크고 환경마다 경로가 다름 → `.gitignore`로 제외, `requirements.txt`만 공유

4. **새 터미널을 열 때마다 활성화해야 하는가?**
   - **YES** — 활성화는 터미널 세션 단위, 창을 새로 열 때마다 다시 활성화 필요

5. **가상환경 활성화 여부 확인 방법?**
   - 프롬프트의 `(.venv)` 확인 또는 `which python` (macOS/Linux) / `where python` (Windows)

venv 오류 트러블슈팅

- **자주 발생하는 venv 오류와 해결 방법**

  - `(.venv)` **표시가 안 나타날 때**
    - 활성화 스크립트가 정상 실행되지 않았거나, 쉘 테마가 프롬프트 수정을 차단
    - 확인: `python -c "import sys; print(sys.prefix)"` → 경로가 가상환경 폴더인지 체크

  - `pip install` **했는데** `ModuleNotFoundError`**가 날 때**
    - 설치한 터미널과 실행 터미널이 다르거나, 가상환경이 비활성화 상태에서 설치
    - 현재 활성화된 환경을 다시 확인

  - **venv 폴더를 다른 곳으로 복사했을 때**
    - 실행 파일 내의 경로가 이전 위치를 가리켜 오류 발생
    - **기존 폴더를 삭제하고 새 위치에서 다시 생성**하는 것이 정석

  - **VSCode에서 인터프리터를 못 찾을 때**
    - `Ctrl+Shift+P` → **"Python: Select Interpreter"** → 생성한 가상환경 경로 직접 선택

Part 5. pip + requirements.txt 실전

pip install 실행 시 내부적으로 일어나는 일

- `pip install` **은 단순 파일 다운로드를 넘어 여러 단계를 거침**
  - **의존성 확인 및 해결**
    - 설치하려는 패키지가 요구하는 다른 패키지(의존성) 확인
    - **의존성 트리(Dependency Tree)** 분석 → 버전 충돌 검사
  - **패키지 다운로드 및 빌드**
    - PyPI(Python Package Index)에서 Wheel 또는 소스 배포판 다운로드
  - **설치 위치 결정**
    - 현재 활성화된 Python 인터프리터의 `site-packages` **디렉토리**에 설치
    - venv 사용 중이라면 해당 가상환경 폴더 내부에만 설치 → 시스템 환경과 격리
  - **실행 파일 생성**
    - 명령줄 도구가 포함된 패키지라면 `bin` / `Scripts` 폴더에 실행 파일 생성

pip 기본 명령어

```
# ROS Humble 환경 호환 패키지 설치 (Python 3.10 기반)
pip install numpy==1.26.4

# 현재 환경의 패키지 목록을 파일로 저장
pip freeze > requirements.txt

# requirements.txt로 패키지 일괄 설치
pip install -r requirements.txt
```

requirements.txt 역할과 팀 협업

- **팀 프로젝트에서 모든 개발자가 동일한 소프트웨어 환경을 갖추는 것이 필수**
  - **환경 재현성 보장**
    - 설치된 모든 패키지와 버전을 기록한 목록
    - 다른 팀원이 프로젝트를 클론했을 때 동일한 환경을 즉시 재구성
  - **격리된 환경 관리**
    - 가상환경과 함께 사용 → 팀원마다 다른 프로젝트를 진행해도 버전 충돌 방지
  - **배포 자동화**
    - 서버/클라우드 배포 시 이 파일을 기반으로 필요한 라이브러리를 한 번에 설치
  - **버전 충돌 해결 방법**
    - **가상환경(`venv`) 사용**: 프로젝트별 독립된 폴더에 패키지 격리
    - **버전 고정(Pinning)**: `requirements.txt`에 정확한 버전 명시

로봇 개발 주요 Python 패키지

- **로봇 공학은 다학제적 분야 — Python은 센서 처리부터 고수준 로직까지 광범위하게 활용**

패키지	설치 명령	주요 용도
NumPy	`pip install numpy`	운동학(Kinematics) 계산, 좌표계 변환, Lidar 점군 데이터 처리
OpenCV	`pip install opencv-python`	객체 인식, 얼굴/색상 감지, Aruco 마커 트래킹
Matplotlib	`pip install matplotlib`	모터 전류, PID 오차 신호 실시간 텔레메트리 시각화
PySerial	`pip install pyserial`	Arduino, 모터 드라이버 등과의 시리얼 통신

pip freeze vs pip list 차이

- **두 명령어 모두 설치된 패키지를 보여주지만 용도가 다름**

  - `pip freeze` — 기록/재현 중심
    - `package==version` 형식으로 출력 → `requirements.txt`에 그대로 저장 가능

  - `pip list` — 가독성 중심
    - 설치된 패키지를 읽기 쉬운 표 형식으로 표시
    - 최신 버전 확인이나 현재 환경 파악에 유용

```
# pip list 결과 (가독성 중심)
Package    Version
---------- -------
numpy      1.26.4
pip        23.1.2

# pip freeze 결과 (기록/재현 중심)
numpy==1.26.4
```

자주 발생하는 pip 오류와 해결법

- **로봇 개발 환경에서 자주 겪는 오류들**

1. **ModuleNotFoundError**
   - 패키지가 설치되지 않았거나 잘못된 환경에서 실행 중
   - `pip install <패키지명>` 설치 또는 VSCode 인터프리터 확인

2. **Broken Environment (Broken)**
   - `pyvenv.cfg` 파일 유실 또는 Python 실행 파일 경로 변경
   - **가상환경 폴더를 삭제하고 다시 생성**

3. **Permission Denied (권한 오류)**
   - 시스템 Python 영역에 설치 시도 시 발생
   - 가상환경 사용 또는 `--user` 옵션 추가

4. **SerialException (pyserial 관련)**
   - 로봇 연결 포트 충돌 또는 장치 미연결
   - `try-except SerialException` 블록으로 포트 닫고 재연결 시도

Part 6. VSCode 설정 + Python 연동

Python 확장 설치부터 인터프리터 선택까지

- **VSCode에서 Python 개발 환경을 설정하는 단계별 절차**

1. **준비물 설치:** Python 3 + VSCode 설치

2. **Python 확장 설치:** 확장 뷰(`Ctrl+Shift+X`) → 'Python' 검색 → Microsoft 확장 설치

3. **워크스페이스 열기:** 프로젝트 폴더 생성 후 `code .` 명령 또는 파일 > 폴더 열기

4. **가상환경 생성:** 명령 팔레트(`Ctrl+Shift+P`) → **'Python: Create Environment'** → **'Venv'** 선택
   → 인터프리터 선택 → `.venv` 폴더 생성

5. **인터프리터 선택:** 명령 팔레트 → **'Python: Select Interpreter'** → 생성한 가상환경 선택
   → 상태 표시줄에 Python 버전 + 가상환경 이름 표시

venv와 VSCode 연동 원리

- **VSCode가 가상환경을 탐지하고 관리하는 방식**
  - **격리된 구조**
    - venv는 시스템 기본 Python과 분리된 전용 디렉토리 트리 생성
    - 자체 Python 인터프리터 + `site-packages` 디렉토리 포함
  - **환경 자동 탐색**
    - VSCode는 **PET(Python Environment Tool)** 이라는 Rust 기반 바이너리 사용
    - 워크스페이스 내의 `.venv`, `venv` 폴더 또는 시스템 `PATH`를 자동 스캔
  - **통합 터미널 가상환경 자동 활성화**
    - 활성화 시 쉘의 `PATH` 환경 변수가 수정됨
    - 설정에 따라 터미널이 열릴 때 자동으로 활성화 스크립트 실행
    - `shellStartup`: 쉘 시작 시 스크립트로 즉시 활성화
    - `command`: 터미널 열린 직후 `source .venv/bin/activate` 자동 실행

로봇 개발에 유용한 VSCode 확장 프로그램

- **로봇 소프트웨어 개발을 위한 추천 확장**

확장 프로그램	주요 용도
Python	OpenCV, NumPy 등의 IntelliSense(자동 완성) 제공
Python Debugger	센서 데이터 스트림, 로봇 내부 상태 변수 실시간 검사
WSL	Windows 환경에서 Linux 기반 ROS 환경 사용 시 필수

입문자용 단축키 TOP 10

- **VSCode에서 가장 자주 쓰이는 단축키**

단축키	기능
`Ctrl+Shift+P`	명령 팔레트 열기 (인터프리터 선택 등)
`Ctrl+Shift+X`	확장(Extensions) 보기 열기
`Ctrl+Shift+```	새 통합 터미널 열기
`Ctrl+S`	현재 파일 저장
`Shift+Enter`	선택 영역 코드를 터미널에서 즉시 실행
`F5`	디버깅 시작 또는 계속 실행
`Shift+F5`	디버깅 중지
`F9`	현재 줄에 중단점(Breakpoint) 설정/해제
`F10`	코드 한 줄씩 실행 (함수 내부 진입 안 함)
`F11`	함수 내부로 들어가기 (Step Into)

자주 발생하는 Python 환경 문제와 해결법

- **VSCode 환경 설정 시 자주 겪는 문제**
  - **ModuleNotFoundError**
    - 가상환경 활성화 후 `pip install <패키지명>` 또는 환경 관리자 UI에서 직접 설치
  - **환경이 목록에 나타나지 않음**
    - `workspaceSearchPaths` 설정에 경로 추가 또는 **'Refresh All Environment Managers'** 실행
  - **"(broken)" 환경 표시**
    - `pyvenv.cfg` 파일 누락 또는 실행 파일이 유효하지 않을 때 발생
    - 가상환경을 **삭제하고 다시 생성**하는 것이 가장 확실한 해결책
  - **터미널 자동 활성화 실패**
    - 설정 변경 후 기존 터미널을 닫고 새로 열어야 적용
    - Windows의 경우 실행 정책(PowerShell) 문제로 스크립트 실행 제한 가능 → 권한 설정 확인

Part 7. 종합 실습 시나리오

로봇 프로젝트 환경 구성 — 명령어 시퀀스

```
# [1] 프로젝트 폴더 생성 및 이동
mkdir robot_project && cd robot_project

# [2] Git 저장소 초기화
git init

# [3] 가상환경(venv) 생성 — 프로젝트 전용 독립 Python 환경
python3 -m venv --system-site-packages .venv

# [4] 가상환경 활성화
source .venv/bin/activate

# [5] 필수 로봇 라이브러리 설치
pip install numpy==1.21.5

# [6] 설치된 패키지 목록 저장
pip freeze > requirements.txt

# [7] VSCode 실행 (현재 폴더를 워크스페이스로 열기)
code .
```

정상 실행 시 터미널 출력 예시

- 각 명령어 실행 후 확인해야 할 출력 메시지

  - `git init` 실행 후:

```
Initialized empty Git repository in /path/to/robot_project/.git/
```

  - 가상환경 활성화 후 (가장 중요!):
    - 터미널 프롬프트 가장 왼쪽에 `(.venv)` 가 표시되어야 함

```
(.venv) user@computer:~/robot_project$
```

정상 실행 시 터미널 출력 예시

- 각 명령어 실행 후 확인해야 할 출력 메시지
  - `pip install` 성공 시:

```
Successfully installed numpy-x.x.x
```

  - `python --version` 확인 시:

```
Python 3.10.12
```

자주 발생하는 오류 상황 및 디버깅 체크리스트

- **입문자가 가장 자주 겪는 5가지 상황**

1. **Windows 가상환경 활성화 권한 오류**
   - 증상: "이 시스템에서 스크립트를 실행할 수 없으므로..." 빨간 경고
   - 해결: PowerShell 관리자 권한 →
     `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`

2. **ModuleNotFoundError (설치했는데 없다고 뜰 때)**
   - 설치한 가상환경과 VSCode 인터프리터가 불일치
   - `Ctrl+Shift+P` → `Python: Select Interpreter` → **'.venv': venv** 선택 확인

3. **'python'은 내부 또는 외부 명령이 아닙니다**
   - `python3 -m venv .venv` 또는 `py -m venv .venv` 로 시도

4. `(.venv)` **표시가 사라짐** — 터미널 세션마다 매번 다시 활성화 필요

5. **Git 명령어가 작동하지 않음** — git 설치 후 터미널 재시작

실습 완료 확인 체크포인트

- **성공적으로 환경을 구성했다면 다음 4가지를 모두 만족해야 함**

  - [ ] **가상환경 활성화 여부**: 터미널 프롬프트 왼쪽에 `(.venv)` 표시
  - [ ] **인터프리터 설정**: VSCode 오른쪽 하단 파이썬 버전 옆에 가상환경 이름(`.venv`) 표시
  - [ ] **의존성 파일 생성**: 프로젝트 폴더 내 `requirements.txt`에 `numpy` 목록 확인
  - [ ] **코드 실행 테스트**: 아래 코드를 실행했을 때 오류 없이 결과 출력

```python
# test_robot.py
import numpy as np
# NumPy가 정상 로드되면 아래 연산 결과가 출력됩니다.
print(f"로봇 프로젝트 준비 완료! NumPy 버전: {np.__version__}")
```

VSCode의 **'Run Python File'** 버튼(오른쪽 위 재생 아이콘)을 눌러 하단 터미널에서 결과를 확인하세요.

Part 8. 퀴즈 + 복습 문제

개념 확인 O/X 문제 (1~5)

- **개념 확인 O/X 문제** (1/2)

1. `git commit` 을 실행하면 내 코드가 즉시 GitHub에 업로드된다.
2. 가상환경(venv)의 주요 목적은 프로젝트별 라이브러리 버전을 독립적으로 관리하기 위함이다.
3. VSCode의 Python 확장은 프로젝트 내 `.venv` 가상환경을 자동으로 인식할 수 있다.
4. 가상환경을 활성화하면 터미널 프롬프트 앞에 `(.venv)` 표시가 나타난다.
5. `git add` 명령어는 변경된 파일을 스테이징 영역(Staging Area)으로 이동시킨다.

개념 확인 O/X 문제 (6~10)

- **개념 확인 O/X 문제** (2/2)

6. 가상환경 폴더(`.venv`)는 용량이 크고 재생성 가능하므로 Git 저장소에 포함하지 않는 것이 원칙이다.

7. VSCode에서 중단점(Breakpoint)을 설정하면 코드 실행 중 해당 라인에서 일시 정지하여 변수 값을 확인할 수 있다.

8. `pip freeze > requirements.txt` 명령어는 현재 설치된 라이브러리 목록을 파일로 저장해준다.

9. `git init` 은 이미 존재하는 GitHub 리포지토리를 내 컴퓨터로 다운로드할 때 사용하는 명령어이다.

10. `git add` 후 파일을 수정하면, 반드시 다시 `git add` 를 해야 수정된 내용이 커밋에 포함된다.

빈칸 채우기 문제

- **핵심 명령어 빈칸 채우기**

1. 수정한 파일을 스테이징 영역에 추가할 때:
   `git ( ) .`

2. 가상환경 폴더를 생성할 때:
   `python -m ( ) .venv`

3. `requirements.txt`에 적힌 라이브러리들을 한꺼번에 설치할 때:
   `pip ( ) -r requirements.txt`

4. 커밋 메시지와 함께 변경 사항을 기록할 때:
   `git commit ( ) "메시지 입력"`

5. VSCode에서 가상환경을 사용하기 위해 팔레트에서 검색하는 메뉴:
   `Python: Select ( )`

시나리오형 문제

- **상황에 맞는 정확한 조치를 선택하세요**

  - **시나리오 1:** 새 로봇 제어 프로젝트를 시작하려고 합니다. 코드의 수정 이력을 기록하고 싶을 때 **가장 먼저** 해야 할 작업은?

  - **시나리오 2:** `OpenCV` 라이브러리를 설치했는데, **다른 프로젝트에 영향을 주지 않고** 이 프로젝트에서만 사용하고 싶습니다. 어떤 순서로 조치해야 할까요?

  - **시나리오 3:** VSCode에서 코드 실행 시 `ModuleNotFoundError`가 발생합니다. 터미널에는 `(.venv)` 표시가 있는데 무엇이 문제일까요?

실습 체크리스트

- **성공적인 환경 구성 자가 진단 — 모두 체크하세요!**

  - [ ] VSCode와 최신 버전의 Python이 설치되어 있는가?
  - [ ] VSCode에 'Python' 확장 프로그램이 설치되어 있는가?
  - [ ] 프로젝트 폴더를 생성하고 VSCode로 열었는가?
  - [ ] `git init`을 통해 로컬 저장소를 초기화했는가?
  - [ ] `python -m venv .venv` 명령어로 가상환경을 생성했는가?
  - [ ] 사용 중인 OS에 맞는 스크립트로 가상환경을 활성화했는가?
  - [ ] VSCode 인터프리터 설정에서 생성한 가상환경(`.venv`)을 선택했는가?
  - [ ] `pip install numpy` 등 필요한 패키지를 하나 이상 설치해 보았는가?
  - [ ] `hello.py` 파일을 만들고 VSCode의 **Run** 버튼으로 실행해 보았는가?
  - [ ] 코드 수정 후 `git add`와 `git commit`을 통해 첫 스냅샷을 남겼는가?